EMBEDDED SYSTEM PROJECT · ATmega128A

LCD 시계 & 계산기

LCD1602, DS1307 활용 RTC 시계 & 4x4 Keypad 사칙연산 계산기

박상현

00 CONTENTS

목차

발표 순서 및 주요 내용

01

개요

프로젝트 목적 및 개발 환경

02

H/W

회로 구성, Part List, 회로도

03

S/W

전체 FSM, LCD/RTC/계산기 구현

04

동작 영상

실제 동작 데모

05

구현 후기

트러블슈팅 및 배운 점

01 개요 · OVERVIEW

프로젝트 목적

1

텍스트 LCD 제어 원리 습득

4-bit 인터페이스 초기화 및 명령어 체계를 직접 구현하며 타이밍 기반 하드웨어 제어를 학습

2

I2C(2-wire) 프로토콜 실습

DS1307 RTC와의 통신을 통해 START/ACK/STOP 등 I2C 버스 시나리오를 ATmega128 하드웨어 I2C 통신 방법으로 직접 구현

3

문자열 파싱 알고리즘 적용

중위 표기법 수식을 후위 표기법 수식으로 변환하여 결과값을 도출하는 계산기를 구현

최종 동작 요약

평소 : 시계 모드
연도-월-일-요일 / 시-분-초로 표현

전환 시 : 계산기 모드
키패드와 버튼을 통해 입력을 받음

BUTTON3 : 시계 ↔ 계산기 토글
1분간 무입력 시 자동으로 시계 모드 전환

시계 모드 : BUTTON2로 년/월/일/요일/시/분/초 설정 가능
BUTTON0/1로 값 증감, DS1307에 즉시 저장

01 개요 · OVERVIEW

개발 환경

H/W 플랫폼 및 S/W 개발 도구

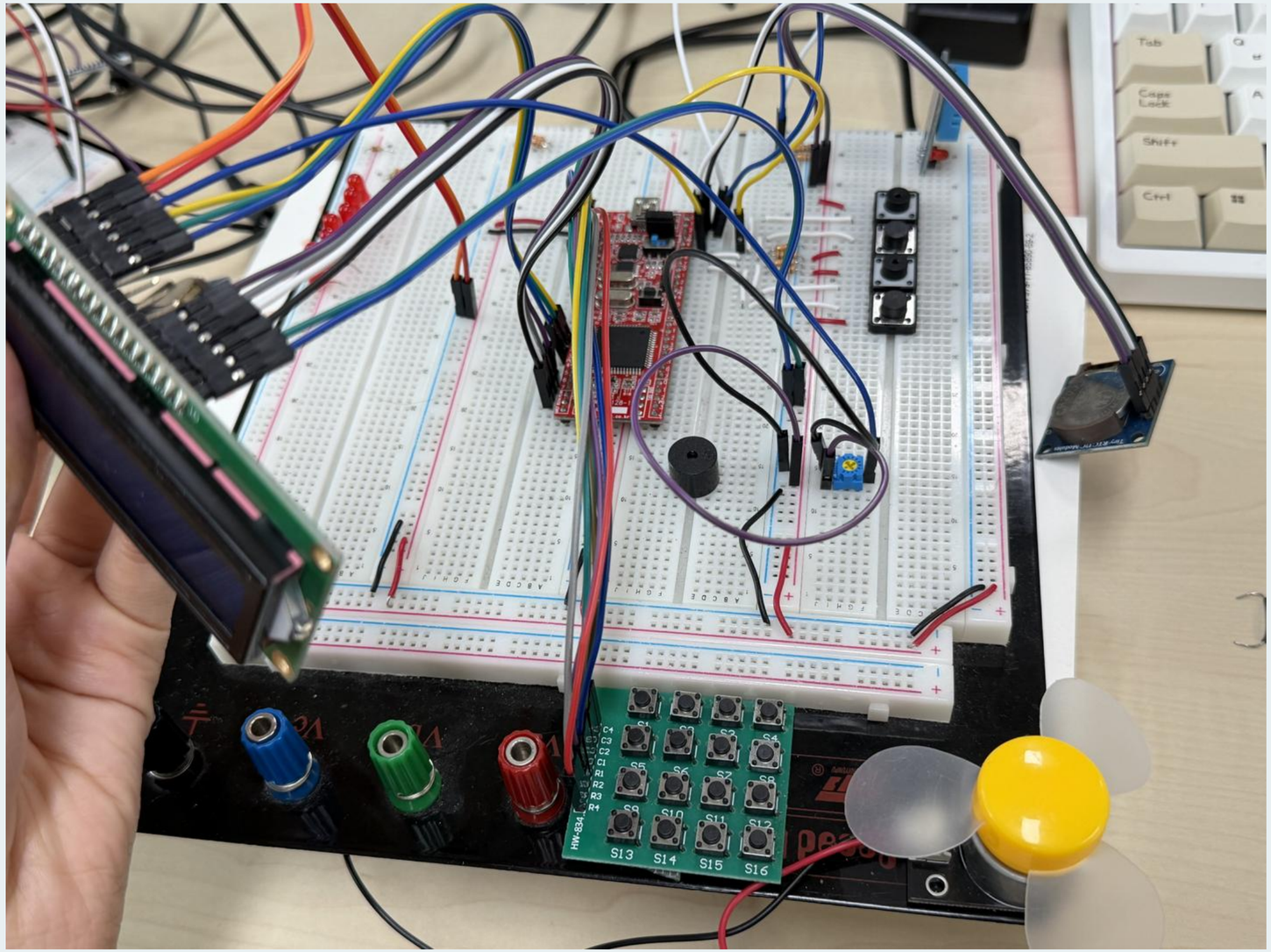
HARDWARE

MCU	ATmega128
LCD	LCD1602
RTC	DS1307 (64×8 Serial Real-Time Clock)
입력장치	4×4 Matrix Keypad + 4 Push Button
통신	I2C

SOFTWARE

IDE	Atmel Studio
컴파일러	AVR-GCC
핵심 모듈	LCD.c · RTC.c · cal.c · keypad.c · button.c
제어 방식	Timer1 CTC 1초 인터럽트 + Non-blocking Polling 기반 메인 루프

외관 사진



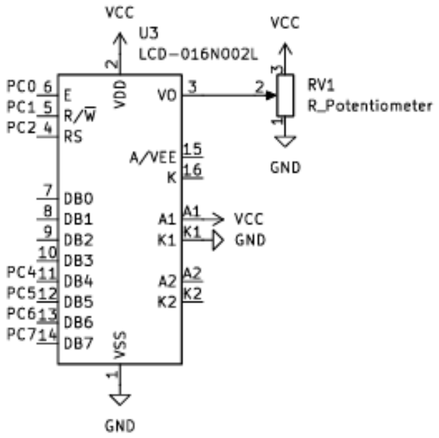
02 H/W · HARDWARE

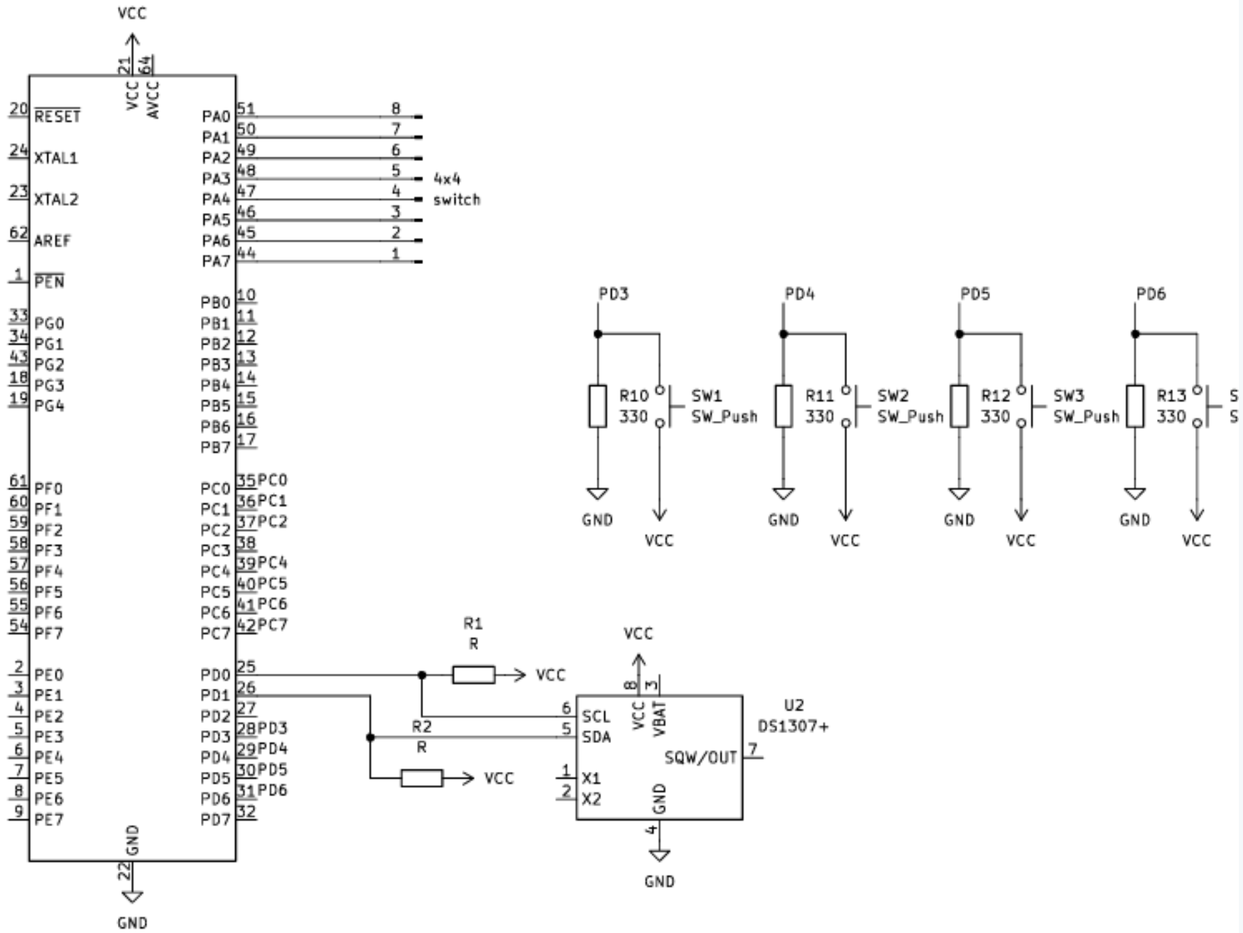
Part List

사용 부품 목록

구분	부품명	역할
MCU	ATmega128	메인 컨트롤러
Display	LCD1602	계산기/시계 화면 출력
RTC	DS1307	실시간 시계, I2C 통신
Input	4×4 Matrix Keypad	숫자/연산자/괄호 입력
Input	Push Button	모드 전환, 시간 설정
Passive	Pull-up Resistor	SDA/SCL 라인 풀업
Passive	가변저항(VR)	LCD 밝기 조절
Passive	저항	LCD 백라이트(LED+) 전류 제한
Power	Breadboard Power / 5V	전원 공급

회로도

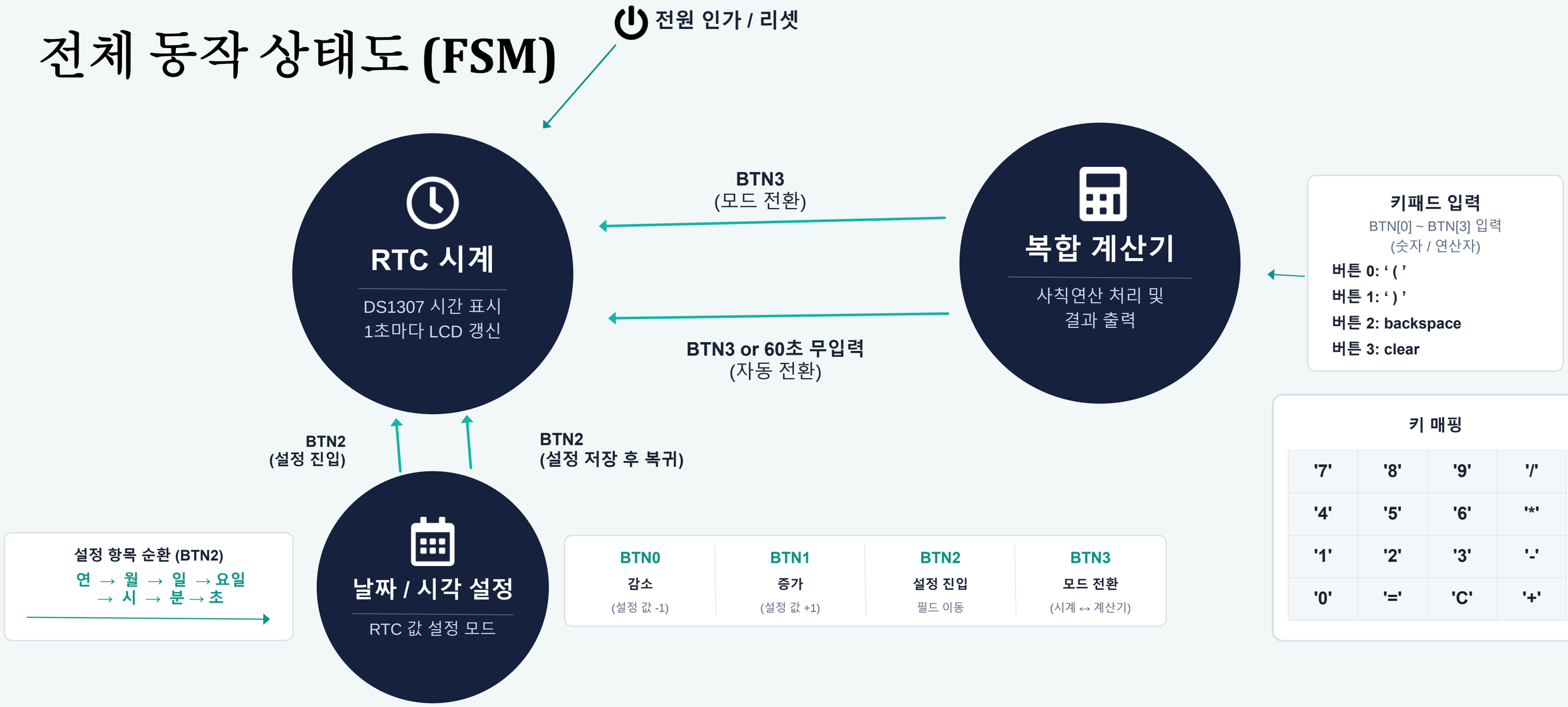




PIN MAP

LCD1602	RS/RW/E	PC0 / PC1 / PC2
LCD1602	D4~D7	PC4 ~ PC7
DS1307	SCL / SDA	PD0 / PD1 (HW TWI)
Keypad	ROW / COL	PA4~7(in) / PA0~3(out)
Button0~3	BTN0~3	PD3 / PD4 / PD5 / PD6

전체 동작 상태도 (FSM)



키 매핑

'7'	'8'	'9'	'/'
'4'	'5'	'6'	'*'
'1'	'2'	'3'	'-'
'0'	'='	'C'	'+'

3.2.1 S/W · DS1307 RTC (I2C)

main 코드

```
int main(void)
{
        char lcd_buff[64];
        rtc_time_t t, edit_t;

        // main 초기화 로직
        init_main(lcd_buff, &t, &edit_t);

        stdout = &OUTPUT; // printf가 동작할 수 있도록 stdout을 설정
        sei(); // 전역(대문) interrupt 허용

        // ds1307에 초기 시간 셋팅
        ds1307_set_time(&t);

    while (1)
    {
    }
}
```

```
while (1)
    {
                // 각종 count 체크
                check_count();

                // 버튼 입력 제어
                button_control(&edit_t);

                // 모드에 따른 lcd 제어
                lcd_control(lcd_buff, &t, &edit_t);
    }
```

```
ISR(TIMER0_OVF_vect)
{
        TCNT0 = 6; // TCNT0 6 ~ 256 : 250개 펄스 count하기 위해
        ms_count++;
}
```

설계 포인트 ISR, while 내부 로직은 함수를 통해 간결하게 작성

3.2.1 S/W · LCD DRIVER

LCD1602 — Datasheet 핵심 요약

- 16문자 × 2라인의 텍스트 공간을 제공하는 범용 디스플레이 장치
- 내부 HD44780 호환 제어칩 포함
- 간결한 명령 체계로 문자 표출

구성	16 Characters × 2 Lines, 5×7 Dot Font
인터페이스	8-bit 병렬 (본 프로젝트는 4-bit 모드로 D4~D7만 사용)
제어 신호	RS(데이터 선택), R/W(읽기/쓰기), E(Enable strobe)
전원	VDD 5V, VEE는 가변저항으로 음영(밝기) 조절
구동 전류	0.2mA(Typ.) / 0.4mA(Max) — 외부 클록 기준

본 프로젝트 적용 사항

8-bit 데이터 라인 중 상위 4비트(D4~D7)만 사용하는 4-bit 모드를 적용

AC CHARACTERISTICS (Write Mode)

tC (E Cycle)	≥ 1000 ns
tPW (E Pulse Width)	≥ 450 ns
tSP2 (Data Setup)	≥ 195 ns
tHD2 (Data Hold)	≥ 10 ns

주요 명령어 (Instruction Set)

```
0x01  Clear Display  (1.64ms)
0x28  Function Set — 4bit,2Line,5×7
0x0C  Display ON / Cursor,Blink OFF
0x06  Entry Mode — 커서 우측 이동
```

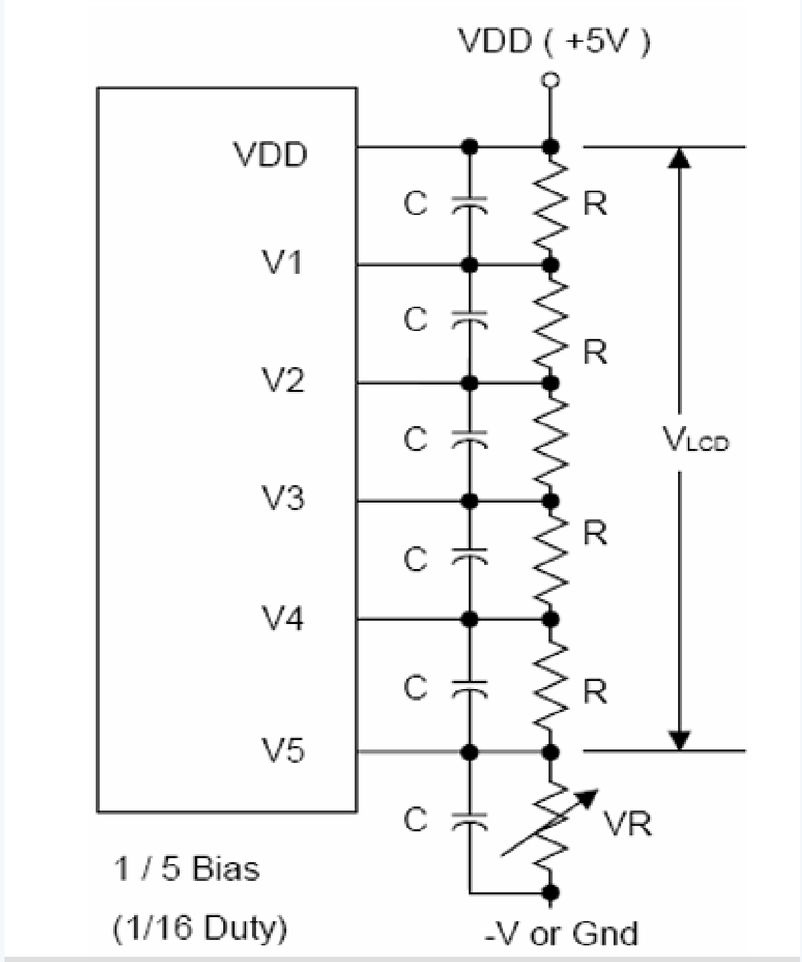
3.2.1 S/W · LCD DRIVER

LCD1602 — Datasheet 핵심 요약

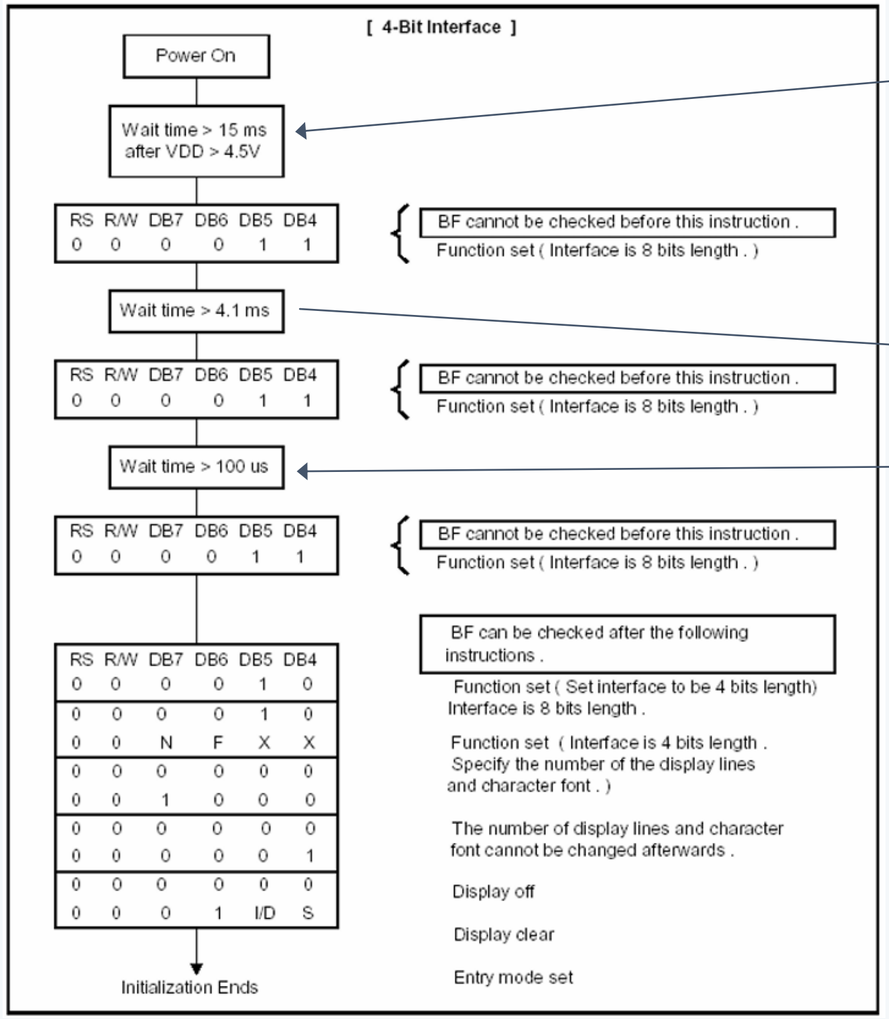
```
VSS - ground
VCC - +5V
VEE - lcd 명암 조절 -> 가변 저항(VR) 연결
RS  - 0: INSTRUCTION, 1: DATA
R/W - 0: WRITE, 1: READ
```

17. INTERFACE

PIN NO.	SYMBOL	DESCRIPTION	FUNCTION
1	VSS	GROUND	0V (GND)
2	VCC	POWER SUPPLY FOR LOGIC CIRCUIT	+5V
3	VEE	LCD CONTRAST ADJUSTMENT	
4	RS	INSTRUCTION/DATA REGISTER SELECTION	RS = 0 : INSTRUCTION REGISTER RS = 1 : DATA REGISTER
5	R/W	READ/WRITE SELECTION	R/W = 0 : REGISTER WRITE R/W = 1 : REGISTER READ
6	E	ENABLE SIGNAL	
7	DB0	DATA INPUT/OUTPUT LINES	8 BIT: DB0-DB7
8	DB1		
9	DB2		
10	DB3		
11	DB4		
12	DB5		
13	DB6		
14	DB7		
15	LED+	SUPPLY VOLTAGE FOR LED+	+5V
16	LED-	SUPPLY VOLTAGE FOR LED-	0V



LCD 초기화 시퀀스 상세




```c
void LCD_init(void)
{
    _delay_ms(50);                          // 초기 구동 대기 시간

    // PORTC: 데이터 핀(PC4~PC7) 출력 설정
    LCD_DATA_DDR |= 0xF0;

    // PORTC: 제어 핀(RS, RW, E) 출력 설정
    LCD_CONTROL_DDR |= (1 << RS_PIN) | (1 << RW_PIN) | (1 << E_PIN);
    LCD_CONTROL_PORT &= ~(1 << RW_PIN);     // RW = 0 고정 (쓰기 모드)

    // 데이터시트 기준: 4비트 모드 진입을 위한 하드웨어 초기화 시퀀스
    LCD_DATA_PORT = (LCD_DATA_PORT & 0x0F) | 0x30;
    LCD_pulse_enable();
    _delay_ms(5);

    LCD_DATA_PORT = (LCD_DATA_PORT & 0x0F) | 0x30;
    LCD_pulse_enable();
    _delay_us(150);

    LCD_DATA_PORT = (LCD_DATA_PORT & 0x0F) | 0x30;
    LCD_pulse_enable();
    _delay_ms(2);

    // 최종 4비트 인터페이스 설정 명령 (상위 니블만 전송)
    LCD_DATA_PORT = (LCD_DATA_PORT & 0x0F) | 0x20;
    LCD_pulse_enable();
    _delay_ms(2);

    // 소프트웨어 설정 (4비트 모드, 2라인 출력, 5x7 도트 폰트)
    LCD_write_command(COMMAND_4_BIT_MODE);

    // Display ON, Cursor OFF, Blink OFF
    LCD_write_command(0x0C);

    LCD_clear();                            // 화면 초기화

    // Entry Mode Set (커서 우측 이동, 화면 고정)
    LCD_write_command(0x06);
}
```

3.2.1 S/W · LCD DRIVER

LCD 초기화 시퀀스 상세

Datasheet [4-Bit Interface] 초기화 흐름 → LCD_init() 구현 매핑

단계	동작	대기시간/비고	코드
Power On	전원 인가 후 대기	VDD > 4.5V 이후 15ms 이상 대기	`_delay_ms(50)`
① Function Set	0x3 상위 3회 전송 (8bit 모드로 강제 리셋)	1차 전송 후 4.1ms 대기	`0x30 write + _delay_ms(5)`
② Function Set	0x3 상위 재전송	100us 대기	`0x30 write + _delay_us(150)`
③ Function Set	0x3 상위 3번째 전송 (BF 체크 불가 구간 종료)	정상 명령 간격만 확보	`0x30 write + _delay_ms(2)`
④ 4-bit 모드 진입	상위 0x2 전송 → 이후 4-bit 인터페이스로 동작	표준 명령 간격	`0x20 write + _delay_ms(2)`
⑤ Function Set	N(라인수)=1, F(폰트)=0 → 0x28 (2 Line, 5×7)	이후 라인수/폰트 변경 불가	`LCD_write_command(0x28)`
⑥ Display OFF/ON 설정	Display, Cursor, Blink 제어 → 0x0C	-	`LCD_write_command(0x0C)`
⑦ Display Clear	화면 지우고 AC=0 (1.64ms)	-	`LCD_clear()`
⑧ Entry Mode Set	I/D=1, S=0 → 0x06 (쓰기 후 커서 우측 이동)	-	`LCD_write_command(0x06)`

※BF(Busy Flag)를 확인할 수 없는 구간이므로, 데이터시트 권장 대기시간을 그대로 소프트웨어 delay로 대체 구현

3.2.1 S/W · LCD DRIVER

LCD Driver — 핵심 코드 구현

4-bit 니블 분할 전송과 좌표 계산 로직

```
// 4비트 모드: 1바이트를 상위→하위 니블 순서로 2번 전송
void lcd_write_data(uint8_t data)
{
        if (data != ' ') lcd_len++;

        LCD_PORT_CONTROL |= 1 << RS_PIN; // 문자 출력에서 RS는 1
        LCD_PORT_DATA = data & 0xF0; // 상위 4비트
        lcd_pulse_enable();
        LCD_PORT_DATA = data << 4 & 0xF0; // 하위 4비트
        lcd_pulse_enable();

        if (lcd_len == 16) lcd_goto_xy(1, 0);
        _delay_ms(2);
}
```

```
// RS 비트로 명령/데이터 레지스터 구분
void lcd_write_command(uint8_t command)
{
        LCD_PORT_CONTROL &= ~(1 << RS_PIN); // 명령어 실행에서 RS는 0
        LCD_PORT_DATA = command & 0xF0; // 상위 4비트
        lcd_pulse_enable();
        LCD_PORT_DATA = command << 4 & 0xF0; // 하위 4비트
        lcd_pulse_enable();

        _delay_ms(2);
}
```

```
// DDRAM 주소 매핑: 1행=0x00~, 2행=0x40~
void lcd_goto_xy(uint8_t row, uint8_t col)
{
        uint8_t address = 0x40 * row + col;
        uint8_t command = 0x80 + address;
        lcd_write_command(command); // 커서 이동
}
```

핵심 포인트 시프트 연산자를 활용해 4-bit 모드 구현

3.2.2 S/W · DS1307 RTC (I2C)

DS1307 — Datasheet 핵심 요약

64×8 Serial Real-Time Clock, 2-wire(I2C) Interface

슬레이브 주소	0x68 (7bit) = 1101000, R/W 비트 추가로 8bit 전송
레지스터 맵	0x00~0x07 : 시계/캘린더 + 제어 · 0x08~0x3F : 56byte RAM
데이터 형식	BCD(Binary-Coded Decimal) — 상위/하위 니블이 각각 10의 자리/1의 자리
통신 속도	Standard Mode 100kHz 전용 (Fast Mode 미지원)

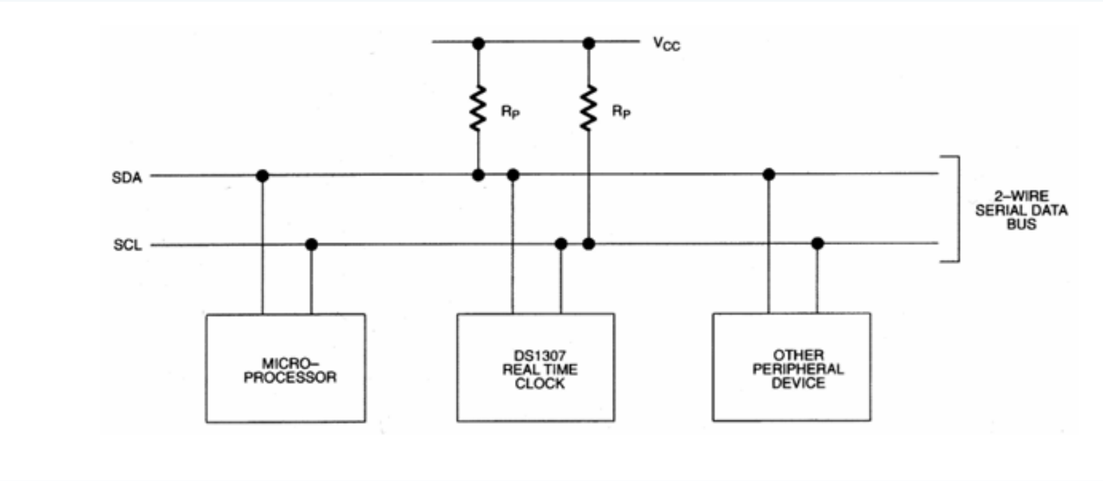
12/24hr 모드

24시간 모드로 고정하여 사용하고, LCD에는 12시간제로 변환해 AM/PM과 함께 표시

타임키퍼 레지스터 (0x00~0x07)

0x00	**SECONDS**	bit7=CH(Clock Halt)
0x01	**MINUTES**	
0x02	**HOURS**	bit6=12/24hr 모드
0x03	**DAY**	1~7 요일
0x04	**DATE**	1~31
0x05	**MONTH**	1~12
0x06	**YEAR**	00~99
0x07	**CONTROL**	OUT,SQWE,RS1,RS0

DS1307 — Datasheet 핵심 요약



SQW/OUT (Square Wave/ Output Driver) - When enabled, the SQWE bit set to 1, the SQW/OUT pin outputs one of four square wave frequencies (1 Hz, 4 kHz, 8 kHz, 32 kHz). The SQW/OUT pin is open drain which requires an external pullup resistor. SQW/OUT will operate with either Vcc or Vbat applied.

핵심 포인트 SDA, SCL에 풀업저항 연결 후 사용

I2C(2-wire) 통신 시나리오

레지스터 Write / Read 프레임 구조

① Write 프레임 — 시간 설정 (ds1307_set_time)



② Read 프레임 1/2 — 레지스터 포인터 설정 + Repeated START



Read 프레임 2/2 — 6byte는 ACK, 마지막 1byte는 NACK 후 STOP



핵심 규칙 S = START, Sr = Repeated START, A = ACK(수신측이 SDA를 LOW로), NA = NACK(마스터가 더 이상 받지 않음을 통보). Write는 STOP으로 종료하고, Read는 마지막 byte에서 NACK을 보내야 슬레이브가 전송을 멈춤

3.2.2 S/W · DS1307 RTC (I2C)

RTC Driver — 핵심 코드 구현

I2C 통신 방법 + BCD 변환 + 시간 Read/Write

```
// TWI 하드웨어 100kHz 설정
#define TWBR_VALUE ((F_CPU/100000UL - 16)/2)
void init_i2c(void)
{
        TWBR = TWBR_VALUE;
        TWSR = 0x00;
}

void i2c_start(void)
{
        TWCR = 1 << TWINT | 1 << TWSTA | 1 << TWEN;
        while (!(TWCR & 1 << TWINT));
}

void i2c_stop(void)
{
        TWCR = 1 << TWINT | 1 << TWSTO | 1 << TWEN;
        _delay_us(10);
}
```

설계 포인트 풀업저항이 없어 슬레이브 응답이 없으면 LCD가 안 켜질 수 있으므로, 풀업저항 연결 필수

```
// BCD ⇔ DEC 변환
uint8_t bcd_to_decimal(uint8_t b) {
  return (bcd >> 4) * 10 + (bcd & 0x0F);
}
uint8_t decimal_to_bcd(uint8_t d) {
  return (decimal / 10) << 4 | decimal % 10;
}
```

```
// 시간 읽기: 레지스터 포인터 = 0 설정 후
void ds1307_get_time(rtc_time_t* t)
{
        i2c_start();
        i2c_write(DS1307_ADDRESS << 1);
        i2c_write(0x00);
        i2c_stop();

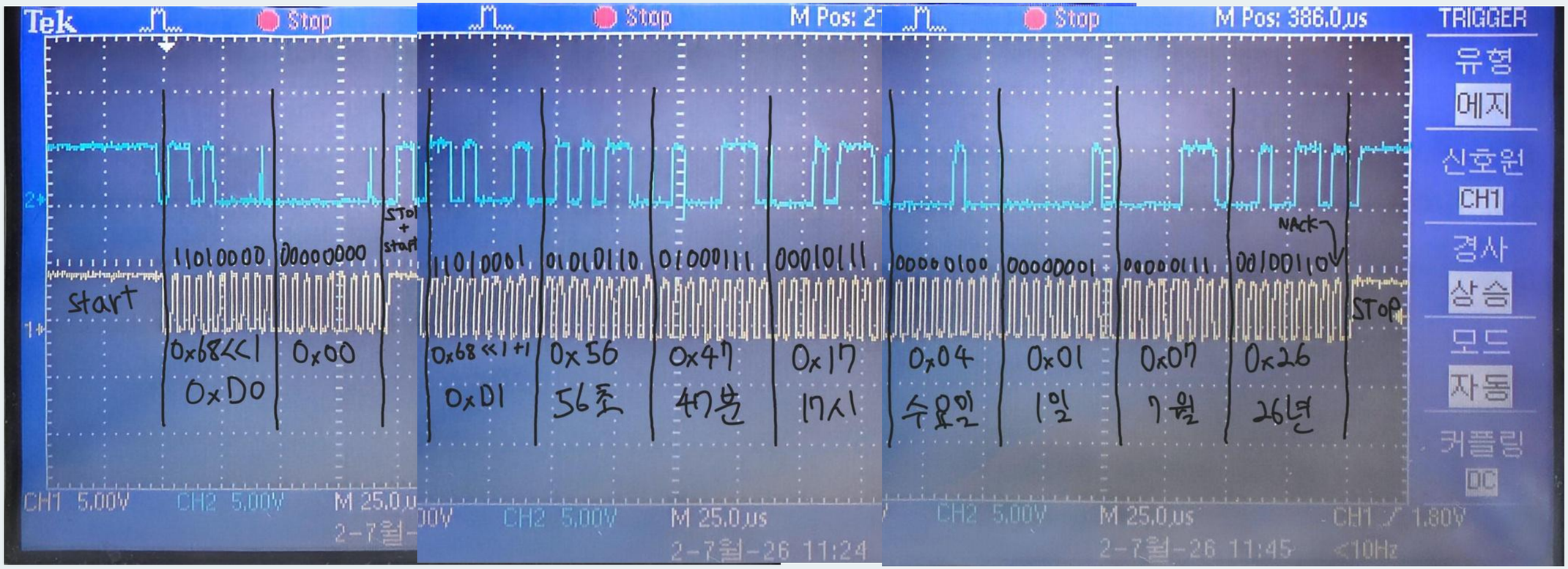
        i2c_start();
        i2c_write(DS1307_ADDRESS << 1 | 1); // i2c 주소 전송(read 모드)

        t->sec = bcd_to_decimal(i2c_read_ack());
        t->year = bcd_to_decimal(i2c_read_nack());
        i2c_stop();
}
```

3.2.2 S/W · DS1307 RTC (I2C)

오실로스코프 동작 파형

SCL / SDA 라인 실측 캡처 (100kHz)



3.2.3 S/W · 계산기

계산기 기본 설계

입력 Sequence 및 상태 흐름 (FSM)

입력 시퀀스 예시 : (1 0 + 2 0) * 1 0 / 1 5 =

1	키 입력	'(' '1' '0' '+' '2' '0' ')' '*' '1' '0' '/' '1' '5' '='
2	동작	각 문자를 inbuff에 저장, LCD 1행과 2행에 표시, 버튼 0과 버튼 1로 괄호 작성, 버튼 2로 백스페이스
3	'=' 입력	후위표기식 변환(outbuff) + 괄호 검사 결과 반환
4	파싱/계산	outbuff를 처음부터 훑으며 우선순위에 맞게 계산
5	결과 출력	계산 결과 LCD 표시, 버튼 3으로 초기화

3.2.3 S/W · 계산기

중위 표기식 -> 후위 표기식 → 계산 (STACK 사용)

예시

중위 표기식: ((222 + 4) * 55) - 100 / 7 * 5 -5 * 10
후위 표기식: 222 4 + 55 * 100 7 / 5 * - 5 10 * -

적용된 코드

// 중위 표기식을 후위 표기식으로 변환
int infix_to_postfix(char* in, char* out)

// 후위 표기식을 계산하여 최종 결과값을 반환
int eval_postfix(char* out)

중위 표기법이란?

연산자(+, -, *, /)를 두 피연산자(숫자)의 **사이에** 배치하여 사람이 직관적으로 이해하기 쉽게 수식을 표현하는 방식

후위 표기법이란?

연산자를 두 피연산자의 **뒤에** 배치하여 괄호나 연산자 우선순위 고려 없이 컴퓨터가 앞에서부터 순차적으로 계산하기 좋도록 표현하는 방식

STACK 이란?

스택은 "가장 나중에 들어간 데이터가 가장 먼저 나오는**(후입선출)**" 구조의 자료구조

계산기 — 핵심 코드 구현

중위 표기식 -> 후위 표기식 변환 + 괄호 검사

```
for (int i = 0; i < length; i++) // 문자열의 길이만큼 반복
{
    if (*in == '(') // 열린 괄호일 때
    {
        open++; // 열린 괄호 갯수 증가
        push(&temp, *in); // 스택에 삽입
    }
    else if (*in == ')') // 닫힌 괄호일 때
    {
        close++; // 닫힌 괄호 갯수 증가
        // 스택이 비어있지 않고, 열린 괄호를 만날 때까지 반복
        while (!is_empty(&temp) && peek(&temp) != '(')
        {
            *out++ = ' '; // 공백 삽입
            *out++ = pop(&temp); // 스택에 들어있던 연산자 삽입
        }
        pop(&temp); // '(' 삭제
    }
```

```
else if (*in == '*' || *in == '/' || *in == '%' || *in == '+' || *in == '-') // 연산자일 때
{
    // 스택이 비어있지 않고, 연산자 우선순위가 작거나 같으면 반복
    while (!is_empty(&temp) && get_priority(*in) <= get_priority(peek(&temp)))
    {
        *out++ = ' '; // 공백 삽입
        *out++ = pop(&temp); // 스택에 들어있던 연산자 삽입
    }
    *out++ = ' '; // 현재 연산자를 쓸 때 공백 삽입
    push(&temp, *in);
}
else if (*in >= '0' && *in <= '9') // 숫자일 때
{
    *out++ = *in; // 숫자 삽입
}

in++; // 다음 문자로 이동
```

```
while (is_empty(&temp) != 1)
{
    *out++ = pop(&temp);
}

*out = '\0';

return open == close;
```

설계 포인트 함수 호출부에서 리턴값을 통해 올바른 식을 줬는지 확인이 가능

계산기 — 핵심 코드 구현

후위 표기식 -> 결과값 반환

```
while (*out != '\0')
{
    if (*out == ' ') // 공백은 패스
    {
        out++;
        continue;
    }

    if (*out >= '0' && *out <= '9') // 숫자를 만났을 때
    {
        int num = 0;

        while (*out >= '0' && *out <= '9') // 숫자면 계속 반복하며 조립 ((두 자릿수 이상 처리를 위함)
        {
            num = (num * 10) + (*out - '0');
            out++;
        }

        push(&eval_stack, num); // 조립한 숫자를 스택에 삽입

        out--; // 반복문 끝에서 이동하므로 중복되지 않도록 이전 문자로 이동
    }
```

```
    // 연산자면 스택에서 2개를 꺼내 계산
    else if (*out == '+' || *out == '-' || *out == '*' || *out == '/' || *out == '%')
    {
        int op2 = pop(&eval_stack);
        int op1 = pop(&eval_stack);

        switch (*out)
        {
            case '+': push(&eval_stack, op1 + op2); break;
            case '-': push(&eval_stack, op1 - op2); break;
            case '*': push(&eval_stack, op1 * op2); break;
            case '/': push(&eval_stack, op1 / op2); break;
            case '%': push(&eval_stack, op1 % op2); break;
        }
    }

    out++; // 다음 문자로
}

return pop(&eval_stack); // 결과 반환
```

설계 포인트 후위 표기식 변환 과정에서 공백이 포함되므로 공백은 반드시 패스해야 함

동작 영상 - 시계



등 전환

동작 영상 - 계산기



전환

구현 후기

트러블슈팅 경험 및 배운 점

1	**I2C 통신 안정화**	DS1307의 SDA/SCL을 ATmega128A에 바로 연결 시 Floating 상태가 되어 통신이 불가능해지는 현상 발생 -> 풀업 저항 연결
2	**LCD 4bit 초기화**	데이터 시트의 초기화하는 대기시간과 코드를 정확히 지키지 않으면 화면이 깨지는 현상을 겪음 → 데이터 시트의 Figure 12 초기화 시퀀스를 그대로 재현하여 해결
3	**LCD 밝기 조절**	초기 VEE를 그라운드에 연결하니 LCD에 글자가 잘 나오지 않는 현상 발생 -> VEE를 가변 저항에 연결
4	**데이터 시트의 중요성**	대부분의 문제는 데이터 시트를 정확하게 파악하지 못한 것에서 발생 -> 데이터 시트의 중요성을 다시금 체감

감사합니다

LCD1602, DS1307 활용 RTC 시계 & 4x4 Keypad 사칙연산 계산기